# BUSINESS REQUIREMENT DOCUMENT (BRD)

# SISTEM POLA MAKAN SEHAT BERBASIS MOBILE DENGAN REKOMENDASI MENU DAN PERHITUNGAN KALORI HARIAN

## 1. Ringkasan Eksekutif

Business Requirement Document (BRD) ini menjelaskan kebutuhan pengembangan Sistem Pola Makan Sehat menjadi aplikasi Android. Proyek ini merupakan kelanjutan dari situs web yang telah dikembangkan pada semester 4 dalam mata kuliah Pemrograman Web.

Aplikasi seluler direncanakan menggunakan Flutter, sedangkan Laravel dan MariaDB tetap digunakan sebagai *backend* dan basis data. Dengan demikian, situs web dan aplikasi seluler dapat menggunakan data yang sama. Pengguna dapat mengelola profil, memantau kebutuhan kalori, memilih makanan, menyusun rencana makan, mencatat konsumsi, dan mengatur daftar belanja melalui perangkat Android.

Pengelolaan data utama dan hak akses administrator tetap dilakukan melalui panel Filament berbasis web. Pengembangan aplikasi seluler berfokus pada penyesuaian tampilan dan penggunaan fitur untuk layar ponsel tanpa mengganti sistem *backend* yang sudah ada.

## 2. Latar Belakang dan Justifikasi Bisnis

### 2.1 Latar Belakang

Pengelolaan pola makan mencakup perhitungan kebutuhan kalori, pemilihan makanan, penyusunan jadwal makan, pencatatan konsumsi, dan pengaturan kebutuhan belanja. Situs web Sistem Pola Makan Sehat telah dikembangkan untuk membantu pengguna menjalankan kegiatan tersebut dalam satu sistem.

Untuk memperluas akses, sistem akan dikembangkan menjadi aplikasi Android. Aplikasi ini diharapkan memudahkan pengguna memeriksa informasi makanan dan memperbarui catatan harian melalui antarmuka yang sesuai untuk perangkat seluler.

### 2.2 Permasalahan

Penggunaan melalui situs web belum secara khusus menyediakan pengalaman aplikasi Android. Pengguna membutuhkan akses yang praktis untuk memantau konsumsi harian dan mengelola rencana makan maupun daftar belanja.

Selain itu, aplikasi seluler harus menggunakan data dan aturan bisnis yang sama dengan situs web agar tidak terjadi perbedaan pencatatan.

### 2.3 Solusi yang Diusulkan

Solusi yang diusulkan adalah membangun aplikasi Android menggunakan Flutter dengan memanfaatkan *backend* Laravel dan basis data MariaDB yang sudah digunakan oleh situs web.

Sebelum integrasi dilakukan, *endpoint* API yang diperlukan akan diperiksa. Bagian yang belum tersedia atau belum sesuai kebutuhan aplikasi seluler perlu dikembangkan. Panel administrator tetap menggunakan situs web yang sudah ada.

## 3. Tujuan Bisnis

Pengembangan aplikasi seluler bertujuan untuk:

1. Memudahkan pengguna memantau estimasi kebutuhan dan konsumsi kalori harian.
2. Menyediakan akses ke koleksi makanan dan rekomendasi menu.
3. Mendukung penyusunan rencana makan berdasarkan tanggal, waktu, dan porsi.
4. Memudahkan pencatatan makanan yang dikonsumsi, termasuk makanan di luar rencana.
5. Membantu pengguna mengatur daftar belanja dan status pembelian.
6. Mempertahankan konsistensi data antara situs web dan aplikasi seluler.
7. Menyediakan antarmuka Android yang mudah dipahami dan digunakan.

## 4. Ruang Lingkup

Bagian ini menjelaskan fitur yang direncanakan tersedia pada aplikasi seluler serta batas pengembangannya.

### 4.1 Registrasi dan Profil Pengguna

Aplikasi menyediakan fungsi registrasi, masuk, keluar, melihat profil, memperbarui data tubuh, dan mengubah kata sandi. Data tubuh yang digunakan untuk perhitungan kalori meliputi jenis kelamin, usia, tinggi badan, berat badan, dan tingkat aktivitas.

**Autentikasi Google dapat diterapkan apabila integrasi dan konfigurasinya tersedia.**

### 4.2 Perhitungan Kalori dan Dasbor

Sistem menghitung estimasi kebutuhan kalori harian berdasarkan data tubuh dan tingkat aktivitas pengguna. Dasbor menampilkan target kalori, konsumsi aktual, sisa atau kelebihan kalori, informasi protein, karbohidrat, lemak, serta rencana makan hari ini apabila tersedia.

### 4.3 Koleksi Makanan dan Rekomendasi Menu

Pengguna dapat melihat makanan pribadi, makanan publik, dan makanan yang ditetapkan sebagai rekomendasi. Informasi makanan mencakup nama, kategori, porsi, nilai gizi, deskripsi, dan gambar apabila tersedia.

Pengguna juga dapat mencari makanan, menyaring daftar berdasarkan kategori atau jenis akses, serta mengelola makanan miliknya sendiri.

### 4.4 Rencana Makan dan Pencatatan Konsumsi

Pengguna dapat membuat rencana makan berdasarkan tanggal, memilih makanan, menentukan waktu makan dan porsi, serta menambahkan catatan. Status setiap makanan dapat diubah menjadi sudah dikonsumsi atau belum dikonsumsi.

Konsumsi aktual dihitung berdasarkan makanan yang telah ditandai dikonsumsi dan catatan makanan tambahan pada tanggal terkait.

### 4.5 Makanan Tambahan

Pengguna dapat mencatat makanan yang dikonsumsi di luar rencana makan, termasuk waktu, porsi, kalori, zat gizi makro, dan catatan. Data tersebut dimasukkan ke dalam ringkasan konsumsi harian.

Makanan tambahan dapat disimpan sebagai makanan pribadi agar dapat digunakan kembali.

### 4.6 Daftar Belanja

Pengguna dapat menambahkan, mencari, memperbarui, dan menghapus item belanja serta mengubah status pembeliannya. Informasi item dapat mencakup jumlah, satuan, kategori, nilai gizi, dan catatan.

Item yang sudah dibeli dan memenuhi ketentuan sistem dapat digunakan sebagai pilihan makanan. Perubahan status pembelian tidak boleh menghapus rencana makan yang sudah tersimpan.

### 4.7 Pengelolaan Administrator

Administrator tetap menggunakan panel Filament berbasis web untuk mengelola akun, peran dan izin, kategori makanan, data makanan, rekomendasi, serta pengaturan sistem sesuai kewenangannya.

Pembuatan panel administrator baru di aplikasi Android tidak termasuk dalam ruang lingkup utama.

### 4.8 Integrasi Sistem

Aplikasi Flutter berkomunikasi dengan *backend* Laravel melalui API. Laravel menangani autentikasi, validasi, hak akses, aturan bisnis, dan penyimpanan data ke MariaDB.

*Endpoint* yang diperlukan akan diperiksa dan disesuaikan jika belum mendukung kebutuhan aplikasi seluler.

### 4.9 Batasan Pengembangan

Batasan proyek ini adalah:

1. Aplikasi ditujukan untuk Android dan diuji menggunakan emulator Android Studio.
2. Fitur aplikasi mengacu pada fungsi situs web yang telah dibuat.
3. Panel administrator tetap berbasis web.
4. Sistem tidak menyediakan diagnosis medis, terapi, atau rekomendasi klinis.

5. Pemesanan bahan makanan, pembayaran, dan integrasi toko daring tidak termasuk dalam ruang lingkup.
6. Mode luring penuh, sinkronisasi otomatis setelah perangkat kembali daring, pengingat waktu nyata, dan integrasi kalender eksternal tidak termasuk dalam ruang lingkup utama.
7. Autentikasi Google bergantung pada ketersediaan konfigurasi dan integrasinya.

## 5. Pemangku Kepentingan dan Pengguna

### 5.1 Pengguna

Pengguna menggunakan aplikasi untuk mengelola profil, memantau kalori, memilih makanan, menyusun rencana makan, mencatat konsumsi, dan mengatur daftar belanja.

### 5.2 Administrator

Administrator mengelola data utama dan hak akses melalui panel web sesuai kewenangannya.

### 5.3 Pengembang

Pengembang bertanggung jawab membangun aplikasi Flutter, memeriksa dan menyesuaikan API, mengintegrasikan aplikasi dengan *backend*, serta menguji fungsi dan keamanan sistem.

## 6. Persyaratan Fungsional

Persyaratan fungsional berikut menjelaskan fungsi yang harus disediakan oleh aplikasi seluler. Ketersediaan setiap *endpoint* perlu diperiksa sebelum fitur terkait diimplementasikan.

### 6.1 Registrasi dan Autentikasi

- FR-01: Sistem menyediakan registrasi akun dengan validasi kelengkapan data dan keunikan alamat surel.
- FR-02: Sistem menyediakan fungsi masuk menggunakan alamat surel dan kata sandi.
- FR-03: Sistem menyediakan fungsi keluar dan mengakhiri sesi pengguna.
- FR-04: Sistem membatasi akses ke halaman dan data yang memerlukan autentikasi.
- FR-05: Sistem mendukung autentikasi Google apabila integrasinya tersedia**.**

### 6.2 Profil Pengguna

- FR-06: Pengguna dapat melihat profil dan data tubuh miliknya.
- FR-07: Pengguna dapat memperbarui nama, surel, jenis kelamin, usia, tinggi badan, berat badan, dan tingkat aktivitas.
- FR-08: Pengguna dapat mengubah kata sandi melalui mekanisme yang tersedia.
- FR-09: Sistem memvalidasi perubahan profil dan memastikan pengguna hanya dapat mengelola data yang menjadi haknya**.**

### 6.3 Perhitungan Kalori

- FR-10: Sistem menghitung *Basal Metabolic Rate* (BMR) berdasarkan data tubuh pengguna.
- FR-11: Sistem menghitung estimasi kebutuhan kalori harian dengan mengalikan BMR dan faktor aktivitas.
- FR-12: Sistem memperbarui hasil perhitungan ketika data tubuh atau tingkat aktivitas berubah.
- FR-13: Sistem menampilkan target kalori, konsumsi aktual, serta sisa atau kelebihan kalori.
- FR-14: Sistem menampilkan jumlah protein, karbohidrat, dan lemak berdasarkan makanan yang diperhitungkan dalam konsumsi harian.

**Rumus Perhitungan**

Rumus Harris-Benedict yang direvisi adalah:

**Laki-laki**

BMR = 88,4 + (13,4 × berat badan) + (4,8 × tinggi badan) − (5,68 × usia)

**Perempuan**

BMR = 447,6 + (9,25 × berat badan) + (3,10 × tinggi badan) − (4,33 × usia)

Kebutuhan kalori harian dihitung menggunakan rumus:

Kebutuhan kalori harian = BMR × faktor aktivitas

Faktor aktivitas yang digunakan:

- Sangat ringan: 1,20
- Ringan: 1,375
- Sedang: 1,55
- Berat: 1,725

**Hasil perhitungan merupakan estimasi dan bukan diagnosis medis.**

### 6.4 Koleksi Makanan

- FR-15: Sistem menampilkan makanan pribadi, makanan publik, dan makanan rekomendasi sesuai hak akses.
- FR-16: Sistem menampilkan kategori, porsi, deskripsi, nilai gizi, dan gambar makanan apabila tersedia.
- FR-17: Pengguna dapat mencari dan menyaring makanan berdasarkan kategori atau jenis akses.

- FR-18: Pengguna dapat menambahkan makanan pribadi.
- FR-19: Pengguna dapat memperbarui dan menghapus makanan miliknya sendiri.

### 6.5 Rekomendasi Menu

- FR-20: Sistem menampilkan makanan yang ditetapkan administrator sebagai rekomendasi.
- FR-21: Pengguna dapat melihat informasi gizi dan memilih rekomendasi untuk dimasukkan ke rencana makan.
- FR-22: Sistem menghitung nilai gizi makanan berdasarkan data dan jumlah porsi yang dipilih.

### 6.6 Rencana Makan

- FR-23: Pengguna dapat membuat rencana makan untuk tanggal tertentu.
- FR-24: Pengguna dapat menambahkan makanan dengan menentukan waktu makan, jumlah porsi, dan catatan.
- FR-25: Pengguna dapat melihat rencana makan berdasarkan tanggal yang dipilih.
- FR-26: Pengguna dapat menandai makanan sebagai telah dikonsumsi atau membatalkan status tersebut.
- FR-27: Sistem menghitung gizi rencana makan dan konsumsi aktual sesuai status setiap item.
- FR-28: Pengguna dapat menghapus item atau rencana makan sesuai fungsi yang tersedia.

### 6.7 Makanan Tambahan

- FR-29: Pengguna dapat mencatat makanan di luar rencana makan beserta tanggal, waktu, porsi, nilai gizi, dan catatan.
- FR-30: Sistem memasukkan makanan tambahan ke dalam ringkasan konsumsi pada tanggal terkait.
- FR-31: Pengguna dapat menyimpan makanan tambahan sebagai makanan pribadi untuk digunakan kembali.
- FR-32: Penghapusan catatan konsumsi tidak menghapus data makanan pribadi yang telah tersimpan.

### 6.8 Daftar Belanja

- FR-33: Pengguna dapat menambahkan, melihat, mencari, menyaring, memperbarui, dan menghapus item belanja.

- FR-34: Sistem menyimpan jumlah, satuan, kategori, informasi gizi, catatan, dan status pembelian sesuai data yang diisi.
- FR-35: Pengguna dapat mengubah status pembelian item.
- FR-36: Sistem menghubungkan item belanja yang memenuhi ketentuan dengan data makanan.
- FR-37: Sistem menyediakan item yang sudah dibeli dan memenuhi ketentuan sebagai pilihan makanan tanpa menghapus rencana makan yang tersimpan.

### 6.9 Dasbor dan Interaksi Aplikasi

- FR-38: Dasbor menampilkan target kalori, konsumsi aktual, sisa atau kelebihan kalori, serta informasi zat gizi makro.
- FR-39: Sistem menyediakan navigasi menuju fitur utama aplikasi.
- FR-40: Sistem menampilkan indikator pemuatan, pesan validasi, informasi kesalahan, dan konfirmasi hasil tindakan.
- FR-41: Sistem memperbarui informasi pada halaman terkait setelah perubahan data berhasil disimpan.

### 6.10 Integrasi API dan Administrator

- FR-42: Aplikasi mengambil dan mengirim data melalui API Laravel.
- FR-43: *Backend* memeriksa autentikasi, hak akses, dan kepemilikan data pada setiap permintaan yang memerlukan perlindungan.
- FR-44: Perubahan data disimpan melalui *backend* menggunakan MariaDB yang sama dengan situs web.
- FR-45: Administrator tetap mengelola data utama dan pengaturan sistem melalui panel Filament berbasis web.
- FR-46: *Endpoint* yang belum mendukung kebutuhan aplikasi seluler harus disesuaikan atau dikembangkan sebelum fitur terkait dinyatakan selesai.

## 7. Persyaratan Non-Fungsional

### 7.1 Keamanan dan Privasi

Kata sandi harus disimpan dalam bentuk *hash* di *backend*. Akses ke data pribadi dibatasi berdasarkan autentikasi, peran, dan kepemilikan akun. Pengguna tidak boleh dapat mengakses data akun lain tanpa kewenangan.

### 7.2 Kemudahan Penggunaan

Antarmuka menggunakan label yang jelas, formulir yang teratur, navigasi yang mudah dikenali, dan pesan kesalahan yang dapat dipahami pengguna.

### 7.3 Tampilan dan Kompatibilitas

Tampilan harus menyesuaikan ukuran layar Android yang menjadi target pengujian. Elemen penting tidak boleh terpotong, bertumpuk, atau sulit digunakan. Pengujian dilakukan melalui emulator Android Studio sesuai konfigurasi versi Android proyek.

### 7.4 Keakuratan Perhitungan

Perhitungan kebutuhan kalori harus mengikuti rumus Harris-Benedict yang direvisi dan faktor aktivitas yang ditetapkan. Nilai gizi dihitung berdasarkan data makanan dan jumlah porsi yang tersimpan.

### 7.5 Konsistensi Data

Situs web dan aplikasi seluler menggunakan *backend* serta basis data yang sama. Perubahan data harus mengikuti aturan bisnis *backend* agar hasil pencatatan tetap konsisten.

### 7.6 Kinerja dan Konektivitas

Aplikasi menampilkan indikator pemuatan saat mengambil data dari peladen. Jika permintaan gagal karena jaringan atau peladen, aplikasi harus menampilkan pesan yang sesuai dan tidak menganggap data berhasil tersimpan sebelum menerima konfirmasi dari *backend*.

### 7.7 Pemeliharaan

Kode Flutter, komunikasi API, dan *backend* Laravel perlu memiliki pembagian tanggung jawab yang jelas agar perubahan antarmuka seluler tidak mengganggu aturan bisnis situs web.

### 7.8 Keamanan Komunikasi

Komunikasi pada lingkungan produksi menggunakan HTTPS. Untuk pengembangan lokal, alamat peladen dan konfigurasi jaringan emulator harus disesuaikan dengan lingkungan yang digunakan.

## 8. Arsitektur Tingkat Tinggi

Sistem terdiri atas beberapa komponen yang saling terhubung:

1. Aplikasi seluler: Flutter digunakan untuk menampilkan halaman autentikasi, dasbor, profil, makanan, rencana makan, pencatatan konsumsi, dan daftar belanja.
2. API: Menjadi penghubung antara aplikasi seluler dan *backend* untuk mengambil maupun menyimpan data.
3. Backend: Laravel menangani validasi, autentikasi, hak akses, perhitungan kalori, dan aturan bisnis.
4. Autentikasi API: Laravel Sanctum digunakan sesuai konfigurasi *backend* untuk mendukung autentikasi *token*.
5. Pengelolaan data: Eloquent ORM digunakan oleh Laravel untuk berinteraksi dengan model data.

6. Basis data: MariaDB menyimpan data akun, profil, makanan, rencana makan, konsumsi, daftar belanja, dan data pendukung lainnya.
7. Panel administrator: Filament tetap digunakan melalui situs web untuk mengelola data utama.
8. Lingkungan pengembangan: Flutter dan Android Studio digunakan untuk aplikasi seluler, sedangkan Docker dapat digunakan untuk menjalankan layanan *backend* dan basis data.

Alur komunikasi sistem:

**Flutter → API Laravel → Logika Bisnis Laravel → MariaDB**

Respons dari *backend* dikirim kembali melalui API untuk ditampilkan pada aplikasi. Situs web dan aplikasi seluler menggunakan sumber data yang sama, tetapi memiliki antarmuka dan hak akses sesuai perannya.

**9. Model Data Ringkas**

Model data aplikasi mengacu pada struktur situs web yang sudah dibuat. Penyesuaian hanya dilakukan jika diperlukan untuk mendukung integrasi API.

| Entitas | Fungsi |
|---|---|
| Pengguna | Menyimpan akun, profil, data tubuh, dan tingkat aktivitas. |
| Kategori Makanan | Mengelompokkan makanan untuk memudahkan pencarian. |
| Makanan | Menyimpan informasi makanan, nilai gizi, porsi, gambar, kepemilikan, dan status publik atau rekomendasi. |
| Rencana Makan | Menyimpan pemilik dan tanggal rencana makan. |
| Item Rencana Makan | Menyimpan makanan terpilih, waktu makan, porsi, catatan, dan status konsumsi. |
| Catatan Makanan Harian | Menyimpan makanan tambahan beserta tanggal, waktu, porsi, dan nilai gizinya. |
| Item Daftar Belanja | Menyimpan nama item, jumlah, satuan, kategori, gizi, dan status pembelian. |
| Bahan Makanan | Menyimpan informasi bahan makanan sesuai struktur situs web. |
| Pengaturan Situs | Menyimpan identitas visual dan pengaturan tampilan situs web. |
| Peran dan Izin | Menentukan kewenangan pengguna dan administrator. |
| Catatan Aktivitas | Mencatat aktivitas pengelolaan penting melalui panel administrator. |

Nama tabel, kolom, dan hubungan antarentitas mengikuti implementasi basis data situs web. Struktur tersebut perlu diperiksa sebelum integrasi agar tidak menimbulkan data ganda atau hubungan yang tidak sesuai.

## 10. Alur Proses Pengguna

### 10.1 Registrasi dan Masuk

Pengguna membuat akun apabila belum terdaftar, kemudian masuk menggunakan kredensial yang sesuai. *Backend* memvalidasi permintaan sebelum memberikan akses ke fitur yang memerlukan autentikasi.

### 10.2 Pengisian Profil

Pengguna melengkapi data tubuh dan tingkat aktivitas. Sistem menggunakan data tersebut untuk menghitung estimasi kebutuhan kalori harian.

### 10.3 Pemantauan Konsumsi

Pengguna membuka dasbor untuk melihat target kalori dan konsumsi hariannya. Informasi diperbarui berdasarkan catatan konsumsi dan rencana makan yang relevan.

### 10.4 Pemilihan Makanan

Pengguna mencari makanan melalui koleksi atau memilih rekomendasi. Setelah memeriksa informasi gizi, pengguna dapat memilih makanan untuk rencana makan.

### 10.5 Penyusunan Rencana Makan

Pengguna menentukan tanggal, makanan, waktu makan, porsi, dan catatan. Sistem menyimpan rencana melalui *backend*.

### 10.6 Pencatatan Konsumsi

Setelah makan, pengguna menandai item yang telah dikonsumsi. Jika mengonsumsi makanan di luar rencana, pengguna dapat mencatatnya sebagai makanan tambahan. Sistem memperbarui ringkasan konsumsi berdasarkan data tersebut.

### 10.7 Pengelolaan Daftar Belanja

Pengguna mencatat kebutuhan belanja dan memperbarui status pembelian. Item yang sudah dibeli dan memenuhi ketentuan dapat digunakan sebagai pilihan makanan berikutnya.

### 10.8 Pengelolaan Administrator

Administrator mengelola data utama melalui panel Filament berbasis web. Perubahan yang tersimpan akan digunakan oleh aplikasi seluler sesuai hak akses dan aturan bisnis sistem.

## 11. Teknologi

Teknologi yang digunakan atau direncanakan dalam pengembangan sistem adalah:

| Teknologi | Fungsi |
|---|---|

| | |
|---|---|
| Flutter | Membangun antarmuka aplikasi Android. |
| Dart | Bahasa pemrograman aplikasi Flutter. |
| Android Studio | Pengembangan dan pengujian aplikasi melalui emulator Android. |
| Laravel | Mengelola *backend*, validasi, autentikasi, dan aturan bisnis. |
| PHP | Bahasa pemrograman *backend* Laravel. |
| MariaDB | Menyimpan data utama sistem. |
| Laravel Sanctum | Mendukung autentikasi API sesuai konfigurasi *backend*. |
| API Laravel | Menghubungkan aplikasi seluler dengan *backend*. |
| Filament 3 | Menyediakan panel administrator berbasis web. |
| Livewire dan Blade | Mendukung antarmuka situs web yang sudah ada. |
| Docker | Menjalankan layanan *backend* dan basis data dalam lingkungan pengembangan. |
| Nginx | Menangani layanan web sesuai konfigurasi situs web. |
| Git dan GitHub | Mengelola versi dan penyimpanan kode proyek. |

Sebelum integrasi dilakukan, ketersediaan *endpoint* API, konfigurasi autentikasi, dan kesesuaian teknologi dengan proyek perlu diperiksa.

**12. Asumsi**

Pengembangan aplikasi didasarkan pada asumsi berikut:

1. Pengguna memiliki perangkat Android atau menggunakan emulator Android Studio.
2. *Backend* Laravel dan MariaDB dari proyek situs web dapat dijalankan kembali.
3. Basis data situs web tetap menjadi sumber data utama.
4. Aplikasi dapat mengakses API melalui jaringan yang sesuai.
5. *Endpoint* yang belum tersedia dapat dikembangkan atau disesuaikan.
6. Pengguna memiliki koneksi jaringan ketika memakai fitur yang membutuhkan *backend*.
7. Data tubuh dan informasi gizi yang dimasukkan pengguna sesuai dengan data yang dimiliki.
8. Administrator memeriksa data makanan sebelum menetapkannya sebagai makanan publik atau rekomendasi.
9. Autentikasi Google hanya diterapkan apabila konfigurasi integrasinya tersedia.

10. Perhitungan kalori merupakan estimasi, bukan pengganti konsultasi tenaga medis atau ahli gizi.
11. Panel administrator tetap menggunakan situs web yang sudah ada.

**13. Risiko dan Mitigasi**

| Risiko | Dampak | Mitigasi |
|---|---|---|
| *Backend* tidak dapat diakses | Aplikasi gagal mengambil atau menyimpan data. | Memastikan layanan Docker dan Laravel berjalan serta alamat peladen sesuai dengan konfigurasi emulator. |
| *Endpoint* API belum tersedia | Fitur seluler tidak dapat menggunakan data yang diperlukan. | Memeriksa *endpoint*, menguji respons API, dan mengembangkan bagian yang belum memenuhi kebutuhan. |
| Kegagalan autentikasi | Pengguna tidak dapat masuk atau kehilangan akses. | Menyesuaikan mekanisme autentikasi Flutter dan Laravel serta menangani *token* yang tidak berlaku. |
| Data tidak konsisten | Perubahan pada aplikasi dan situs web tidak tercatat dengan benar. | Menggunakan basis data yang sama dan menguji proses penyimpanan serta pembaruan dari kedua antarmuka. |
| Kesalahan perhitungan kalori | Informasi kebutuhan kalori tidak sesuai dengan hasil yang diharapkan. | Menggunakan rumus dan faktor aktivitas yang sama dengan situs web serta menguji hasil menggunakan data pembanding. |
| Kesalahan masukan | Data tidak valid masuk ke sistem. | Melakukan validasi pada formulir Flutter dan *backend* Laravel. |
| Tampilan tidak sesuai | Elemen aplikasi terpotong atau sulit digunakan. | Menggunakan tata letak yang menyesuaikan layar dan menguji halaman melalui emulator. |
| Gangguan jaringan | Permintaan gagal ketika koneksi terputus atau peladen tidak merespons. | Menampilkan pesan kesalahan dan tidak menyatakan data berhasil disimpan sebelum ada konfirmasi *backend*. |
| Akses tidak sah | Pengguna dapat mencoba membuka data atau fungsi di luar kewenangannya. | Memeriksa autentikasi, peran, dan kepemilikan data pada *backend*. |
| Perbedaan aturan bisnis | Perhitungan konsumsi atau pengelolaan belanja berbeda dari situs web. | Menjadikan *backend* sebagai acuan aturan bisnis dan menguji alur yang sama pada kedua antarmuka. |

**14. Kriteria Penerimaan**

Aplikasi dinyatakan memenuhi kebutuhan apabila hasil pengujian menunjukkan bahwa:

1. Pengguna dapat melakukan registrasi, masuk, dan keluar dengan validasi yang sesuai.
2. Pengguna dapat melihat serta memperbarui profil dan data tubuh miliknya.
3. Sistem menghitung kebutuhan kalori menggunakan rumus dan faktor aktivitas yang ditetapkan.
4. Dasbor menampilkan target kalori, konsumsi aktual, sisa atau kelebihan kalori, dan zat gizi makro dengan benar.
5. Pengguna dapat melihat, mencari, menyaring, dan mengelola makanan pribadi sesuai hak akses.
6. Pengguna dapat melihat rekomendasi dan memasukkannya ke rencana makan.
7. Pengguna dapat membuat dan mengelola rencana makan berdasarkan tanggal, waktu, porsi, dan catatan.
8. Perubahan status konsumsi memperbarui ringkasan harian dengan benar.
9. Pengguna dapat mencatat makanan tambahan dan melihat pengaruhnya terhadap konsumsi harian.
10. Makanan tambahan dapat disimpan sebagai makanan pribadi dan digunakan kembali.
11. Pengguna dapat mengelola item daftar belanja serta memperbarui status pembeliannya.
12. Item belanja yang memenuhi ketentuan dapat digunakan sebagai pilihan makanan tanpa menghapus rencana makan yang sudah tersimpan.
13. Aplikasi dapat mengambil dan menyimpan data melalui API Laravel dengan autentikasi yang sesuai.
14. Data yang diperbarui melalui aplikasi tetap konsisten dengan situs web karena menggunakan *backend* dan basis data yang sama.
15. Pengguna tidak dapat mengakses data akun lain atau fungsi administrator tanpa kewenangan.
16. Administrator tetap dapat mengelola data utama melalui panel web.
17. Aplikasi dapat dijalankan melalui emulator Android Studio tanpa kesalahan yang menghambat fungsi utama.
18. Tampilan dapat digunakan tanpa elemen penting terpotong atau bertumpuk.
19. Aplikasi menampilkan pesan yang sesuai ketika validasi gagal, jaringan bermasalah, atau permintaan API tidak berhasil.
20. *Endpoint* yang diperlukan untuk fitur utama tersedia dan telah diuji.

**15. Diagram Kasus Penggunaan**

Diagram Kasus Penggunaan menggambarkan hubungan antara aktor dan fungsi yang tersedia pada Sistem Pola Makan Sehat.

### 15.1 Aktor Pengguna

Pengguna berinteraksi dengan aplikasi seluler untuk:

1. Melakukan registrasi, masuk, dan keluar.
2. Melihat dan memperbarui profil.
3. Melihat estimasi kebutuhan kalori.
4. Memantau konsumsi harian melalui dasbor.
5. Melihat dan mengelola makanan pribadi.
6. Melihat rekomendasi menu.
7. Membuat dan mengelola rencana makan.
8. Menandai makanan yang telah dikonsumsi.
9. Mencatat makanan tambahan.
10. Mengelola daftar belanja dan status pembelian.

### 15.2 Aktor Administrator

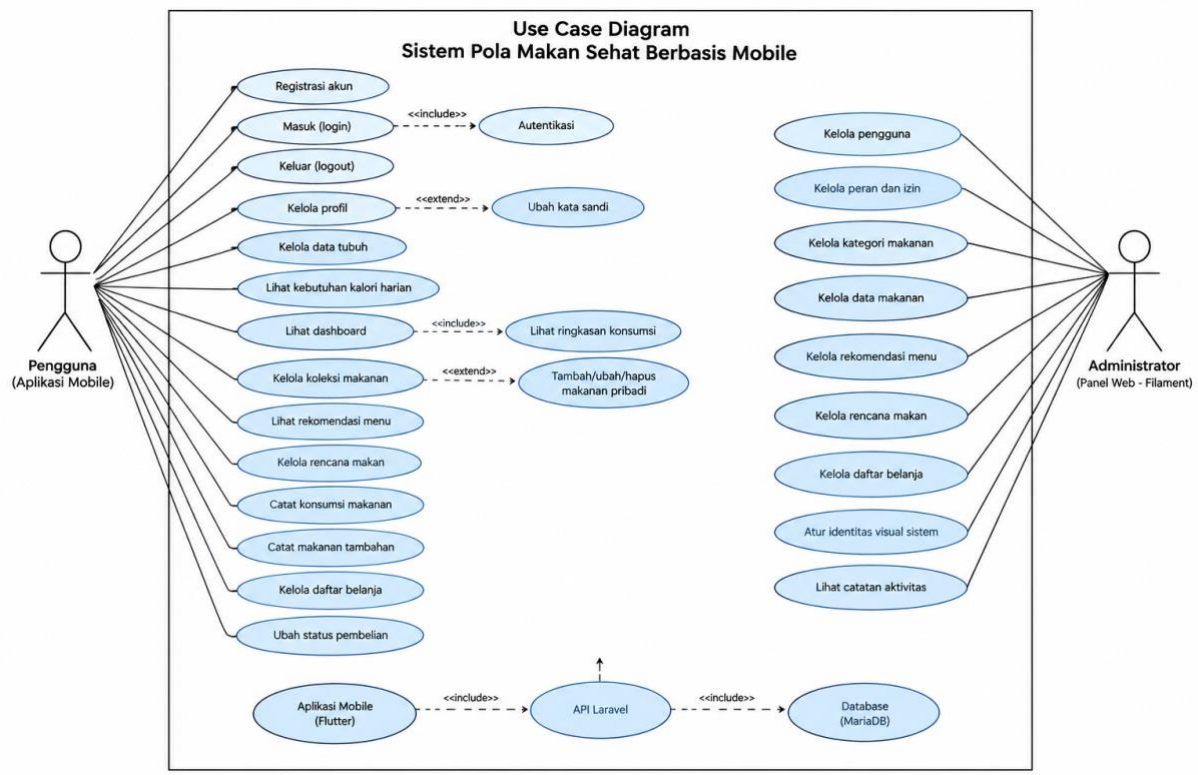
Administrator menggunakan panel Filament berbasis web untuk:

1. Mengelola pengguna, peran, dan izin.
2. Mengelola kategori dan data makanan.
3. Mengelola rekomendasi menu.
4. Mengelola data lain sesuai kewenangannya.
5. Mengatur identitas visual sistem.
6. Memantau catatan aktivitas.

### 15.3 Hubungan Aktor dengan Sistem

Pengguna mengakses fitur utama melalui aplikasi Flutter. Permintaan yang membutuhkan data diproses oleh *backend* Laravel sebelum membaca atau menyimpan data pada MariaDB.

Administrator menggunakan panel web untuk mengelola data utama. Kedua antarmuka menggunakan *backend* dan basis data yang sama, tetapi memiliki fungsi dan kewenangan berbeda.




*Gambar 1. Use Case Diagram Sistem Pola Makan Sehat Berbasis Mobile.*

## Penutup

BRD ini menjadi acuan pengembangan Sistem Pola Makan Sehat dari situs web menjadi aplikasi Android. Pengembangan berfokus pada penyediaan akses melalui Flutter dengan tetap menggunakan *backend* Laravel, basis data MariaDB, dan panel administrator berbasis web yang telah tersedia.

Sebelum implementasi, struktur basis data, aturan bisnis, dan *endpoint* API perlu diperiksa untuk memastikan seluruh fitur seluler dapat terhubung dengan sistem yang sudah ada. Pengujian dilakukan untuk memastikan fungsi berjalan sesuai kebutuhan, data tetap konsisten, dan akses pengguna terlindungi.